



Arquitectura Model-Vista-Controlador

Explicada amb un exemple en Java: el paquet mvc

Què és MVC?

- Patró d'arquitectura que separa l'aplicació en tres responsabilitats
- Model: dades i lògica de negoci
- Vista: presentació i interacció amb l'usuari
- Controlador: coordina el model i la vista
- Objectiu: baix acoblament i alta cohesió
- Cada part es pot canviar, provar i reutilitzar de manera independent

Les tres peces i el seu paper

MODEL

Conté les regles de negoci

No sap res de la vista

No fa cap System.out

Greeting

GreetingService

VISTA

Mostra informació

Llegeix l'entrada d'usuari

No pren decisions

GreetingView

CONTROLADOR

Orquestra el flux

Rep dades de la vista

Crida el model i

retorna el resultat

GreetingController

Estructura del paquet

```
nf1.fullstack.arquitectures.mvc
|-- Main.java
|-- model
|   |-- Greeting.java
|   `-- GreetingService.java
|-- view
|   `-- GreetingView.java
`-- controller
    `-- GreetingController.java
```

- Un subpaquet per cada rol
- Main només construeix i connecta
- Dependències:

Controller -> Model

Controller -> View

Model no depèn de ningú

View no coneix el Model

Model: dades i lògica de negoci

```
public class Greeting {  
    private final String message;  
  
    public Greeting(String message) {  
        this.message = message;  
    }  
    public String getMessage() {  
        return message;  
    }  
}
```

```
public class GreetingService {  
    public Greeting greet(String name) {  
        DayOfWeek dow =  
            LocalDate.now().getDayOfWeek();  
        String salutation;  
        if (dow == DayOfWeek.SATURDAY) {  
            salutation = "Bon cap de setmana";  
        } else if (dow == DayOfWeek.SUNDAY) {  
            salutation = "Bon diumenge";  
        } else { salutation = "Bon dia"; }  
        String msg = String.format(  
            "%s, %s!", salutation, name);  
        return new Greeting(msg);  
    }  
}
```

- Greeting: entitat immutable que guarda el missatge
- GreetingService: decideix la salutació segons el dia de la setmana (regla de negoci)
- El model no imprimeix ni llegeix res: no depèn de la consola

Vista: presentació

```
public class GreetingView {  
    private final Scanner scanner = new Scanner(System.in);  
  
    public String readName() {  
        System.out.print("Introdueix el teu nom (o 'quit' per sortir): ");  
        return scanner.nextLine().trim();  
    }  
  
    public void showGreeting(String greeting) {  
        System.out.println(greeting);  
    }  
  
    public void showGoodbye() {  
        System.out.println("Adéu!");  
    }  
}
```

- Única classe que toca System.in / System.out
- No té lògica: només llegeix i mostra
- Es podria substituir per una vista web o gràfica sense tocar el model

Controlador: coordinació

```
public class GreetingController {  
    private final GreetingView view;  
    private final GreetingService service;  
  
    public GreetingController(GreetingView view, GreetingService service) {  
        this.view = view; this.service = service;  
    }  
    public void run() {  
        String name;  
        while (!(name = view.readName()).equalsIgnoreCase("quit")) {  
            Greeting greeting = service.greet(name);  
            view.showGreeting(greeting.getMessage());  
        }  
        view.showGoodbye();  
    }  
}
```

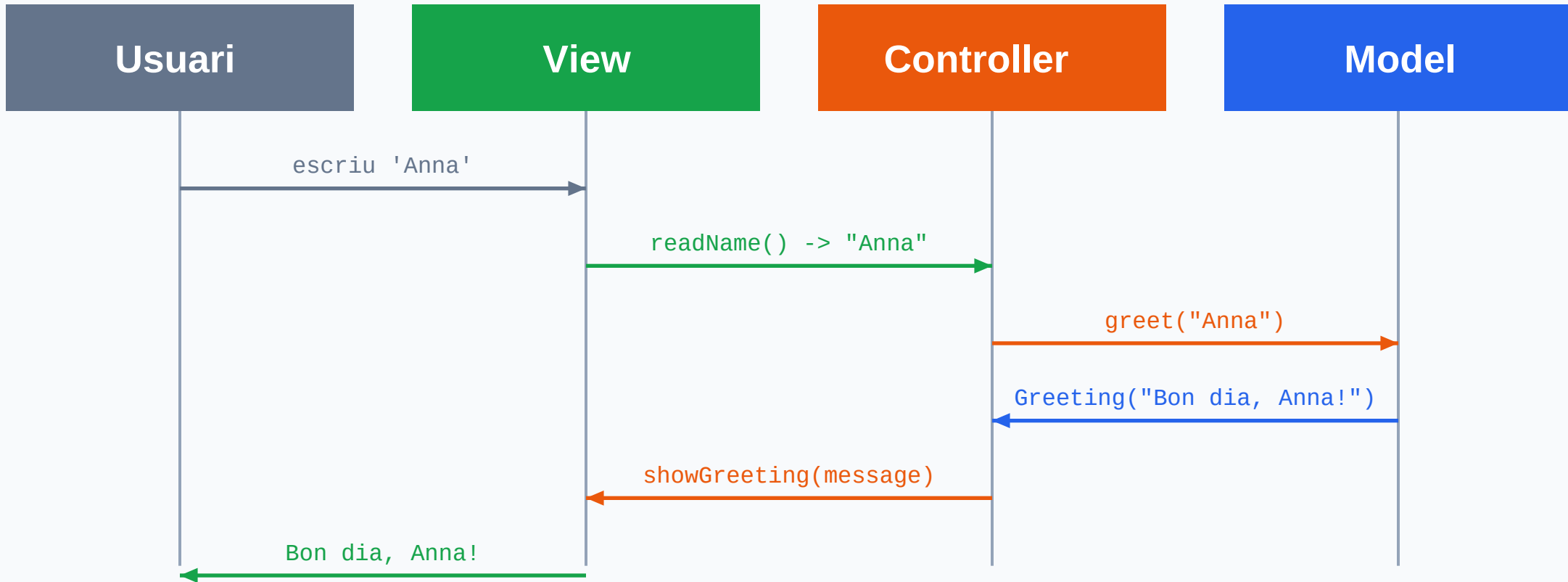
- Rep les dependències pel constructor (injecció de dependències)
- Bucle: llegir nom -> demanar salutació al model -> mostrar-la
- Decideix quan acabar: l'entrada 'quit'

Main: composició de l'aplicació

```
public class Main {  
    public static void main(String[] args) {  
        GreetingView view = new GreetingView();  
        GreetingService service = new GreetingService();  
        GreetingController controller =  
            new GreetingController(view, service);  
        controller.run();  
    }  
}
```

- Crea les tres peces i les connecta (composition root)
- Després cedeix el control a controller.run()
- Canviar de vista o de servei només afecta aquesta classe

Flux d'execució



El bucle es repeteix fins que l'usuari escriu 'quit'; llavors showGoodbye() mostra "Adéu!".

Avantatges i bones pràctiques

- Separació de responsabilitats: cada classe té un únic motiu per canviar
- Testabilitat: GreetingService es prova sense consola
- Substituïble: nova vista (web, GUI) reutilitzant model i controlador
- Treball en equip: es pot desenvolupar cada capa en paral·lel
- Compte: no posar lògica de negoci al controlador ni a la vista
- Compte: el model no ha de dependre de la vista

Resum

Model

Greeting, GreetingService: dades i regles

Vista

GreetingView: entrada i sortida

Controlador

GreetingController: coordina el flux

Gràcies! Preguntes?



Arquitectura Hexagonal

Ports i adaptadors, explicada amb el paquet hexagonal1

Full Stack - NF1

Què és l'arquitectura hexagonal?

- Patró també conegut com a ports i adaptadors (Alistair Cockburn)
- El domini (lògica de negoci) és el centre de l'aplicació
- El domini defineix ports: interfícies del que necessita
- Els adaptadors implementen els ports (consola, fitxer, web, BD...)
- Les dependències apunten sempre cap al domini
- Objectiu: aïllar la lògica de negoci de la tecnologia externa
- Es pot canviar o afegir un adaptador sense tocar el domini

Les tres peces i el seu paper

DOMINI

Conté les regles de negoci
No sap res de la consola
ni del fitxer
Només coneix el port

`Greeting`

`GreetingService`

PORT

Interfície que defineix
què necessita el domini
És un contracte
Cap implementació

`OutputPort`

ADAPTADORS

Implementen el port
Parlen amb el món exterior
Intercanviables
Sense lògica de negoci

`ConsoleOutputAdapter`

`FileOutputAdapter`

`CompositeOutputAdapter`

Estructura del paquet

```
nf1.fullstack.architectures.hexagonal1
|-- Main.java
|-- domain
|   |-- Greeting.java
|   `-- GreetingService.java
|-- application
|   `-- OutputPort.java
`-- adapter
    |-- ConsoleOutputAdapter.java
    |-- FileOutputAdapter.java
    `-- CompositeOutputAdapter.java
```

- domain: el nucli, sense dependències tècniques
- application: conté el port (OutputPort)
- adapter: implementacions concretes del port
- Main només construeix i connecta

Dependències:

GreetingService -> OutputPort

Adapters -> OutputPort

El domini no depèn dels adaptadors

Els adaptadors depenen del port (inversió)

Domini: dades i lògica de negoci

```
public class Greeting {  
    private final String message;  
  
    public Greeting(String message) {  
        this.message = message;  
    }  
    public String getMessage() {  
        return message;  
    }  
}
```

```
public class GreetingService {  
    private final OutputPort output;  
  
    public GreetingService(OutputPort output) {  
        this.output = output;  
    }  
  
    public void greet(String name) {  
        Greeting greeting =  
            new Greeting("Hola, " + name + "!");  
        output.send(greeting.getMessage());  
    }  
}
```

- Greeting: entitat immutable que guarda el missatge
- GreetingService: crea la salutació i l'envia a través del port
- El domini rep el port pel constructor i no sap quin adaptador hi ha darrere

Port: el contracte del domini

```
public interface OutputPort {  
    void send(String message);  
}
```

Per què és important?

Inversió de dependències: el domini
defineix la interfície i els adaptadors
s'hi adapten

- Interfície al paquet application
- Defineix què necessita el domini: enviar un missatge
- No diu com ni on s'envia
- El domini depèn d'aquesta abstracció, no d'una classe concreta

Un sol mètode: `send(String)`

Tants adaptadors com calguin

Adaptadors: implementen el port

```
public class ConsoleOutputAdapter
    implements OutputPort {
    public void send(String m) {
        System.out.println(m);
    }
}
```

```
public class CompositeOutputAdapter
    implements OutputPort {
    private final List<OutputPort> adapters;

    public void send(String message) {
        for (OutputPort a : adapters)
            a.send(message);
    }
}
```

```
public class FileOutputAdapter implements OutputPort {
    private static final String FILE_PATH = "greetings.txt";

    public void send(String message) {
        Files.write(Paths.get(FILE_PATH),
            (message + System.lineSeparator()).getBytes(),
            StandardOpenOption.CREATE, StandardOpenOption.APPEND);
    }
}
```

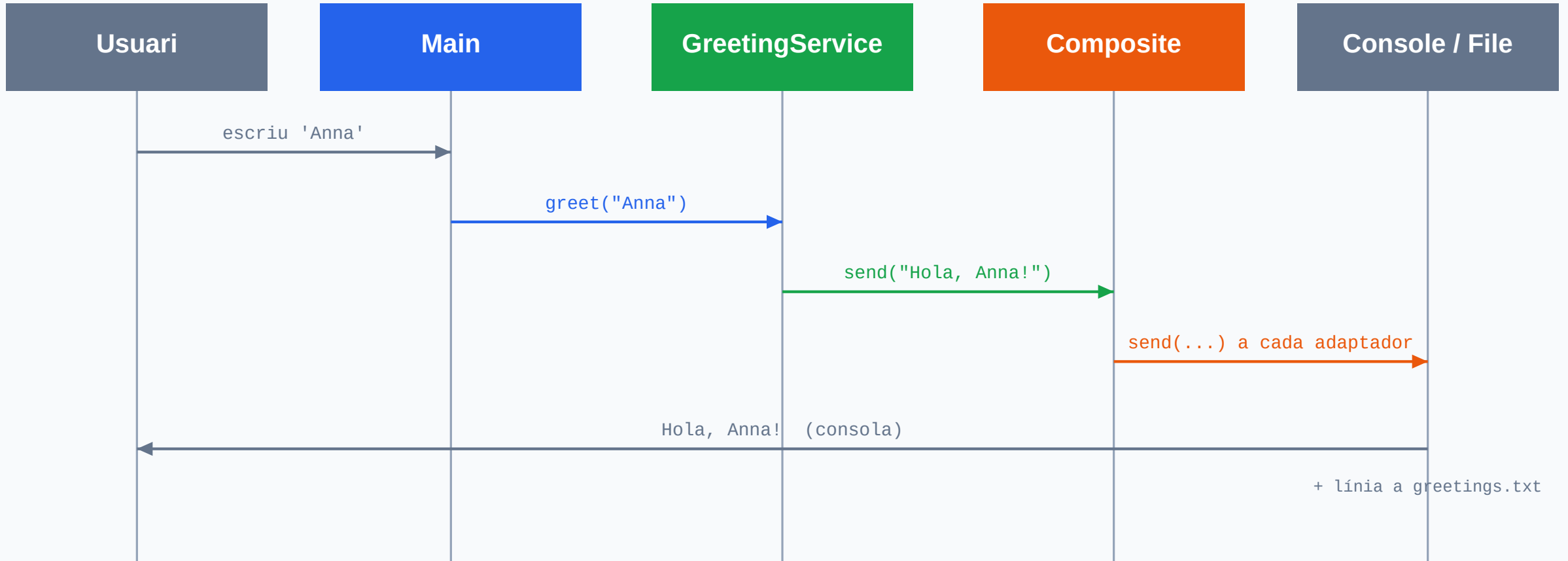
- ConsoleOutputAdapter: escriu per consola
- FileOutputAdapter: afegeix el missatge a greetings.txt
- CompositeOutputAdapter: reenvia a una llista d'adaptadors
- El domini no pot distingir un adaptador d'un altre

Main: composició de l'aplicació

```
public class Main {  
    public static void main(String[] args) {  
        ConsoleOutputAdapter console =  
            new ConsoleOutputAdapter();  
        FileOutputAdapter file =  
            new FileOutputAdapter();  
        CompositeOutputAdapter composite =  
            new CompositeOutputAdapter(List.of(console, file));  
        GreetingService service =  
            new GreetingService(composite);  
  
        Scanner scanner = new Scanner(System.in);  
        System.out.print("Escriu el teu nom: ");  
        service.greet(scanner.nextLine());  
    }  
}
```

- Crea els adaptadors i el servei de domini
- Els connecta: composition root
- Injecta l'adaptador compost al domini
- Llegeix el nom i invoca greet()
- Canviar d'adaptador només afecta Main

Flux d'execució



El domini només coneix OutputPort; el CompositeOutputAdapter envia el missatge a consola i a fitxer.

Hexagonal vs MVC

MVC

- Separa presentació, control i model
- Organitza per rols (capes)
- El controlador coneix model i vista
- Pensat per a la interfície d'usuari

Hexagonal

- Aïlla el domini de l'exterior
- Organitza per ports i adaptadors
- El domini només coneix els seus ports
- Pensat per a entrada i sortida de qualsevol tipus

Avantatges i bones pràctiques

- Domini independent de la tecnologia (consola, fitxer, web, BD)
- Testabilitat: es prova GreetingService amb un port fals
- Extensibilitat: nous adaptadors sense modificar el domini
- Flexibilitat: combinar sortides amb el Composite
- Compte: el domini no ha d'importar classes dels adaptadors
- Compte: no posar lògica de negoci als adaptadors

Resum

Domini

Greeting, GreetingService: dades i regles

Port

OutputPort: contracte de sortida

Adaptadors

Console, File, Composite: implementen el port

Gràcies! Preguntes?